L M U ☀ C M S I 3 8 0 2

LANGUAGES AND AUTOMATA II

HOMEWORK #2 Due: 2025-02-21

Learning Objectives

In this assignment you will demonstrate:

- ✅ The ability to craft a language grammar from a natural language description of a programming language
- ✅ The ability to craft syntax diagrams from a natural language description of a programming language
- ✅ The ability to write parsers using the Ohm library
- ✅ The ability to write test cases for a parser
- ✅ The ability to extend a Node.js project on GitHub while maintaining 100% unit test coverage

Readings, Videos, and Self-Study

Although your team will be turning in only one submission, every team member is individually responsible for each of the following:

- ✅ Do any of the assigned readings and watchings for Homework 1 that you did not get to before.
- ✅ Work through the course notes on Astro, Bella, Carlos, Compilers and Interpreters, and How to Write a Compiler (up to the section entitled *Design the Program Representation*).
- ✅ Read Chapters 1–1.4 of [Mogenson].
- ✅ Read Ch. 3–5 of [Bernhardt].
- ✅ Re-read (or Skim) the Ohm README, the Ohm Tutorial, and the four pages of documentation (Syntax, API, Philosophy, and Patterns & Pitfalls) linked from the main Ohm Documentation.
- ✅ Skim Ch. 2.1–2.5, 2.10, and 3.1–3.6 of [Mogenson]
- ✅ The Wikipedia page on Parsing Expression Grammars.
- ✅ Watch any videos embedded in the relevant course notes.

Submission Instructions

As usual, turn in a single submission for your entire group via BrightSpace. Remember to not simply partition the work among team members, as this reduces your learning.

Your team's BrightSpace submission should be an attached PDF or text file with:

- ✅ The names of every student
- ✅ For each student, an affidavit that each of the readings and watchings above were individually completed
- ✅ Answers to each of the exercises below, numbered and neatly typeset. For problems involving code to write, host your code on Replit or OneCompiler (or similar) and provide the link to the hosting site in your BrightSpace submission.
- ✅ A link to the public GitHub repository hosting your team's project

Exercises

1. Give tiny little Ohm grammars for:
    a. Canadian Postal Codes (make sure to prohibit D, F, I, O, Q, U)
    b. Legal Visa® Card Numbers, *ignoring the Luhn checksums*, i.e., accept `4` + 15 digits or `4` + 12 digits.
    c. Legal MasterCard® Numbers, *ignoring the Luhn checksums*, i.e., accept `51–55` + 14 digits or `2221–2720` + 12 digits.
    d. Strings of Basic Latin letters **except** those strings that are exactly three letters ending with two Latin letter o's, of any case.
    e. Binary numerals divisible by 16.
    f. Decimal numerals in the range 8 through 32, inclusive.
    g. All strings of Unicode letters, except `python`, `pycharm`, or `pyc`.
    h. Floating point constants that are allowed to have an empty fractional part, but whose exponent part is required and can have no more than three digits in the exponent part
    i. Palindromes over the letters a, b, and c, of length 2, 3, 5, or 8
    j. Python string literals. Don't get too fancy here—just translate what you see in the Python Reference linked above into Ohm notation.

    Here is a test for you:

```
import { describe, it } from "node:test"
import assert from "assert"
```

```
import { matches } from "../ohm_exercises.js"

const testFixture = {
  canadianPostalCode: {
    good: ["A7X 2P8", "P8E 4R2", "K1V 9P2", "Y3J 5C0"],
    bad: ["A7X   9B2", "C7E 9U2", "", "Dog", "K1V\t9P2", " A7X 2P8", "A7X 2P8 "],
  },
  visa: {
    good: ["4128976567772613", "4089655522138888", "4098562516243"],
    bad: [
      "43333",
      "42346238746283746823",
      "7687777777263211",
      "foo",
      "π",
      "4128976567772613 ",
    ],
  },
  masterCard: {
    good: [
      "5100000000000000",
      "5294837679998888",
      "5309888182838282",
      "5599999999999999",
      "2221000000000000",
      "2720999999999999",
      "2578930481258783",
      "2230000000000000",
    ],
    bad: [
      "5763777373890002",
      "513988843211541",
      "51398884321108541",
      "",
      "OH",
      "5432333xxxxxxxxx",
    ],
  },
  notThreeEndingInOO: {
    good: ["", "fog", "Tho", "one", "a", "ab", "food"],
    bad: ["fOo", "gOO", "HoO", "zoo", "MOO", "123", "A15"],
  },
```

```
    divisibleBy16: {
      good: ["0", "00", "000", "00000", "00000", "000000", "00000000", "1101000000"],
      bad: ["1", "00000000100", "1000000001", "dog0000000"],
    },
    eightThroughThirtyTwo: {
      good: Array(25)
        .fill(0)
        .map((x, i) => i + 8),
      bad: ["1", "0", "00003", "dog", "", "361", "90", "7", "-11"],
    },
    notPythonPycharmPyc: {
      good: ["", "pythons", "pycs", "PYC", "apycharm", "zpyc", "dog", "pythonpyc"],
      bad: ["python", "pycharm", "pyc"],
    },
    restrictedFloats: {
      good: ["1e0", "235e9", "1.0e1", "1.0e+122", "55e20"],
      bad: ["3.5E9999", "2.355e-9991", "1e2210"],
    },
    palindromes2358: {
      good: [
        "aa",
        "bb",
        "cc",
        "aaa",
        "aba",
        "aca",
        "bab",
        "bbb",
        "ababa",
        "abcba",
        "aaaaaaaa",
        "abaaaaba",
        "cbcbbcbc",
        "caaaaaac",
      ],
      bad: ["", "a", "ab", "abc", "abbbb", "cbcbcbcb"],
    },
    pythonStringLiterals: {
      good: String.raw`''
        ""
        'hello'
        "world"
```

```
      'a\'b'
      "a\"b"
      '\n'
      "a\tb"
      f'\u'
      """abc"""
      '''a'''''''''
      """abc\xdef"""
      '''abc\$def'''
      '''abc\''''`
      .split("\n")
      .map((s) => s.trim()),
    bad: String.raw`
      'hello"
      "world'
      'a'b'
      "a"b"
      'a''
      "x""
      """""""""
      frr"abc"
      'a\'
      '''abc''''
      """`
      .split("\n")
      .map((s) => s.trim()),
  },
}

for (let name of Object.keys(testFixture)) {
  describe(`when matching ${name}`, () => {
    for (let s of testFixture[name].good) {
      it(`accepts ${s}`, () => {
        assert.ok(matches(name, s))
      })
    }
    for (let s of testFixture[name].bad) {
      it(`rejects ${s}`, () => {
        assert.ok(!matches(name, s))
      })
    }
```

```
    })
  }
```

2. Here is a description of a language. Programs in this language are made up of a possibly empty sequence of function declarations, followed by a single expression. Each function declaration starts with the keyword `func` followed by the function's name (an identifier), then a parenthesized list of zero or more parameters (also identifiers) separated by commas, then the body, which is a sequence of one or more expressions separated (NOT terminated) by semicolons with the expression sequence terminated with the keyword `end`. Expressions can be numeric literals, string literals, identifiers, function calls, or can be made up of other expressions with the usual binary arithmetic operators (plus, minus, times, divide) and a unary prefix negation and a unary postfix factorial (`!`). There's a conditional expression with the syntax `y if x else z`. Factorial has the highest precedence, followed by negation, the multiplicative operators, the additive operators, and finally the conditional. Parentheses are used, as in most other languages, to group subexpressions. Numeric literals are non-empty sequences of decimal digits with an optional fractional part and an optional exponent part. String literals delimited with double quotes with the escape sequences `\'`, `\"`, `\n`, `\\`, and `\u{` *hhhhhh* `}` where *xxxxxx* is a sequence of one-to-six hexadecimal digits. Identifiers are non-empty sequences of letters, decimal digits, underscores, at-signs, and dollar signs, beginning with a letter or at-sign, that are not also reserved words. Function calls are formed with an identifier followed by a comma-separated list of expressions bracketed by square brackets. Comments are `--` until the end of the line.

   Write a complete syntactic description of this language in Ohm. (Hint: use the Ohm editor to check your work. When grading, I will copy-paste your submitted solution to BrightSpace into the Ohm editor with my own detailed test suite).

3. Draw syntax diagrams for the phrase rules (not the lexical rules) for the language in the previous exercise.

For Your Project

Continue your compiler project in the public GitHub repository you created in the first assignment. You will be expanding your repo to have the following:

```
.
├── .gitignore
├── README.md
├── LICENSE
├── package.json
├── .prettierrc.json
├── docs
```

```
|   └── ...                   -- image file for your logo
├── examples
|   └── ...                   -- lots of example programs
├── src
|   ├── (yourlanguagename).js
|   ├── (yourlanguagename).ohm
|   ├── compiler.js           -- copied from the course notes
|   ├── core.js               -- this will be empty for now
|   ├── parser.js             -- copied from the course notes
|   ├── analyzer.js           -- contains just the stub function
|   ├── optimizer.js          -- contains just the stub function
|   └── generator.js          -- contains just the stub function
└── test
    ├── compiler.test.js      -- just test that you can invoke the parser
    └── parser.test.js        -- new, test Ohm parsing here
```

Your tasks for this assignment are the following:

- Expand your README as required.
- Construct the files (yourprojectname).js, compiler.js, (yourprojectname).ohm, core.js, analyzer.js, grammar.test.js, and analyzer.test.js as instructed in the How to Write a Compiler notes.
- Design and implement the syntax of your language in an Ohm grammar in your `.ohm` file.
- In your parser.js file, write a parser...a function that will return the Ohm grammar match.
- For the analyzer, optimizer, and generator, though, since these parts are to be written future assignments, you will simply implement them as follows:

```
export default function analyze() {
  throw new Error("Not yet implemented")
}


export default function optimize() {
  throw new Error("Not yet implemented")
}


export default function generate() {
  throw new Error("Not yet implemented")
}
```

- Write parser tests.
- Ensure that Running

```
node src/(yourlanguagename).js (filename) parsed
```

either produces `Syntax is ok` or displays an error.

- Ensure that running

  ```
  npm test
  ```

  runs all your tests, that they all pass, and the coverage report shows 100% coverage.

Grading Rubric

To help you get a good score, here are all the things you will be graded on.

- Exercises (60 pts)
  - ✅ Exercise 1 (30 pts)
  - ✅ Exercise 2 (20 pts)
  - ✅ Exercise 3 (10 pts)
- Compiler Project (40 pts)
  - ✅ Project GitHub repo is in order, exactly as specified above (3 pts)
  - ✅ GitHub repo has no additional files (e.g., no node_modules, no .DS_Store, etc.) (1 pt)
  - ✅ GitHub repo has a README with language name, title, and good description (2 pts)
  - ✅ README has the names of all students (2 pts)
  - ✅ Your project can be cloned (1 pt)
  - ✅ I can run npm test immediately after cloning (1 pt)
  - ✅ All tests pass (3 pts)
  - ✅ Tests have great coverage (you should be able to get 100%, accept nothing less) (2 pts)
  - ✅ The Ohm specification is great and complete (20 pts)
  - ✅ Running from the command line will give the correct answer to the syntax check (5 pts)